Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Пермский государственный аграрно-технологический университет
имени академика Д.Н. Прянишникова»

Факультет экономики и информационных технологий

Кафедра информационных технологий
и программной инженерии

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

(проектно-технологическая практика)

на тему: «**Исследование эффективности алгоритмов сортировки**»

Выполнил:
студент группы ПИНб-21
направления подготовки
09.03.04 Программная инженерия
Власов Дмитрий Алексеевич

Проверил:
доцент кафедры ИТиПИ, к.т.н., доцент
Беляков Андрей Юрьевич

2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1. Анализ алгоритмов сортировки 4
1.1. Алгоритмы сортировки со сложностью n^2 4
1.2. Алгоритмы сортировки со сложностью n*log(n) 4
2. Разработка и исследование эффективности программ сортировки 6
2.1. Разработка программ сортировки 6
2.2. Получение табличных данных и построение графиков 6
ЗАКЛЮЧЕНИЕ 8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 9
ПРИЛОЖЕНИЕ 10

ВВЕДЕНИЕ

Алгоритмы сортировки играют ключевую роль в информатике и программировании, поскольку упорядочивание данных необходимо во многих прикладных задачах — от обработки информации в базах данных и поисковых системах до оптимизации работы программ и систем машинного обучения. Сортировка используется везде, где требуется ускорить поиск, упростить анализ данных или обеспечить корректную обработку структур информации.

Существует множество алгоритмов сортировки, каждый из которых имеет свои особенности, преимущества и ограничения. К наиболее известным и широко применяемым относятся:

Пузырьковая сортировка (Bubble Sort) — простой алгоритм, чаще используемый в образовательных целях из-за наглядности, но малопригодный для работы с большими объёмами данных.

Сортировка выбором (Selection Sort) и **сортировка вставками (Insertion Sort)** — также простые алгоритмы, подходящие для небольших наборов данных.

Быстрая сортировка (Quick Sort) — один из самых эффективных алгоритмов на практике, широко применяемый в стандартных библиотеках многих языков программирования.

Сортировка слиянием (Merge Sort) — алгоритм с гарантированной стабильной производительностью, особенно полезный для сортировки больших объёмов данных.

Пирамидальная сортировка (Heap Sort) — эффективна при работе с приоритетными очередями.

В рамках данной практики будет проведено исследование эффективности различных алгоритмов сортировки с целью выявления их достоинств и недостатков в различных условиях — при разных объёмах данных, типах входных массивов и требованиях к ресурсоёмкости.

1. Анализ алгоритмов сортировки

1.1. Алгоритмы сортировки со сложностью n^2

Сортировка пузырьком (Bubble Sort)

Суть алгоритма: Алгоритм многократно проходит по списку и сравнивает соседние элементы. Если они идут в неправильном порядке (например, слева больше, чем справа), то меняет их местами. Этот процесс повторяется, пока список не отсортируется. С каждым проходом "крупнейшие" элементы "всплывают" в конец массива — отсюда и название.

Почему $O(n^2)$:

На каждой итерации алгоритм выполняет до n сравнений и перемещений. В худшем случае нужно пройтись по массиву n раз, поэтому общее количество операций — примерно $n \times n = n^2$.

Сортировка вставками (Insertion Sort)

Суть алгоритма: Представим, что мы сортируем карты в руке. Мы берём по одной карте и вставляем её в нужное место среди уже отсортированных карт слева. Аналогично, в массиве каждый новый элемент сравнивается с предыдущими, и сдвигается, пока не встанет на своё место.

Почему $O(n^2)$:

Каждый элемент в худшем случае сравнивается со всеми предыдущими (в случае полностью убывающего массива). Для n элементов это даёт $1 + 2 + ... + (n-1) = O(n^2)$ сравнений и сдвигов.

1.2. Алгоритмы сортировки со сложностью n*log(n)

Быстрая сортировка (Quick Sort)

Суть алгоритма: Алгоритм работает по принципу "разделяй и властвуй". Он выбирает опорный элемент (pivot), затем переставляет элементы так, чтобы все меньшие оказались слева от него, а все большие — справа. После этого рекурсивно повторяет процесс для левой и правой части.

Почему $O(n*\log(n))$:

На каждом уровне рекурсии массив делится на две части ($\log(n)$ уровней).

На каждом уровне нужно пройти по всем n элементам, чтобы распределить их по сторонам от опорного.

Получаем $O(n) \times O(\log(n)) = O(n*\log(n))$.

Примечание: В худшем случае (если всегда неудачно выбирать опорный элемент), сложность может быть $O(n^2)$.

Сортировка слиянием (Merge Sort)

Суть алгоритма: Также использует стратегию "разделяй и властвуй". Массив делится пополам, каждая часть сортируется рекурсивно, затем отсортированные части сливаются в один отсортированный массив.

Почему $O(n*\log(n))$:

Глубина рекурсии — log n (каждый раз делим массив пополам).
На каждом уровне требуется O(n) операций на слияние.
Итого: O(n) × O(log n) = O(n*log (n)).
Преимущество: стабильная производительность даже в худшем случае.

2. Разработка и исследование эффективности программ сортировки

2.1. Разработка программ сортировки

В рамках практики были разработаны программы сортировки массивов целых чисел с использованием следующих алгоритмов:

- Сортировка пузырьком (Bubble Sort)
- Сортировка вставками (Insertion Sort)
- Быстрая сортировка (Quick Sort)
- Сортировка слиянием (Merge Sort)

Реализация выполнена на языке программирования Java [2]. Все алгоритмы вынесены в отдельные классы, каждый из которых содержит метод sort, возвращающий информацию о количестве сравнений, перестановок, времени выполнения и использовании памяти.

Пример кода (фрагмент класса сортировки пузырьком):

```
public class BubbleSort {
    public static SortResult sort(int[] array) {
        int n = array.length;
        long comparisons = 0;
        long swaps = 0;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                comparisons++;
                if (array[j] > array[j + 1]) {
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                    swaps++;
                }
            }
        }
        // Возвращается объект SortResult с метриками выполнения
        return new SortResult("BubbleSort", n, comparisons, swaps, elapsedTimeMs,
memoryUsed);
    }
}
```

Для всех алгоритмов реализован единый интерфейс получения статистики через класс SortResult, содержащий поля: имя алгоритма, размер массива, количество сравнений, количество перестановок, время выполнения (в миллисекундах) и объём использованной памяти.

Основной класс Main формирует входные данные, копирует массивы для каждой сортировки и запускает сортировку, собирая результаты и печатая их в виде таблицы в терминале.

2.2. Получение табличных данных и построение графиков

Для проведения экспериментов были сгенерированы случайные входные массивы длиной от 2000 до 20000 элементов с шагом 2000. Для

каждого размера массивов были проведены сортировки всеми четырьмя алгоритмами. Результаты включают количество сравнений, перестановок, время выполнения и память.

Пример таблицы, выводимой программой:

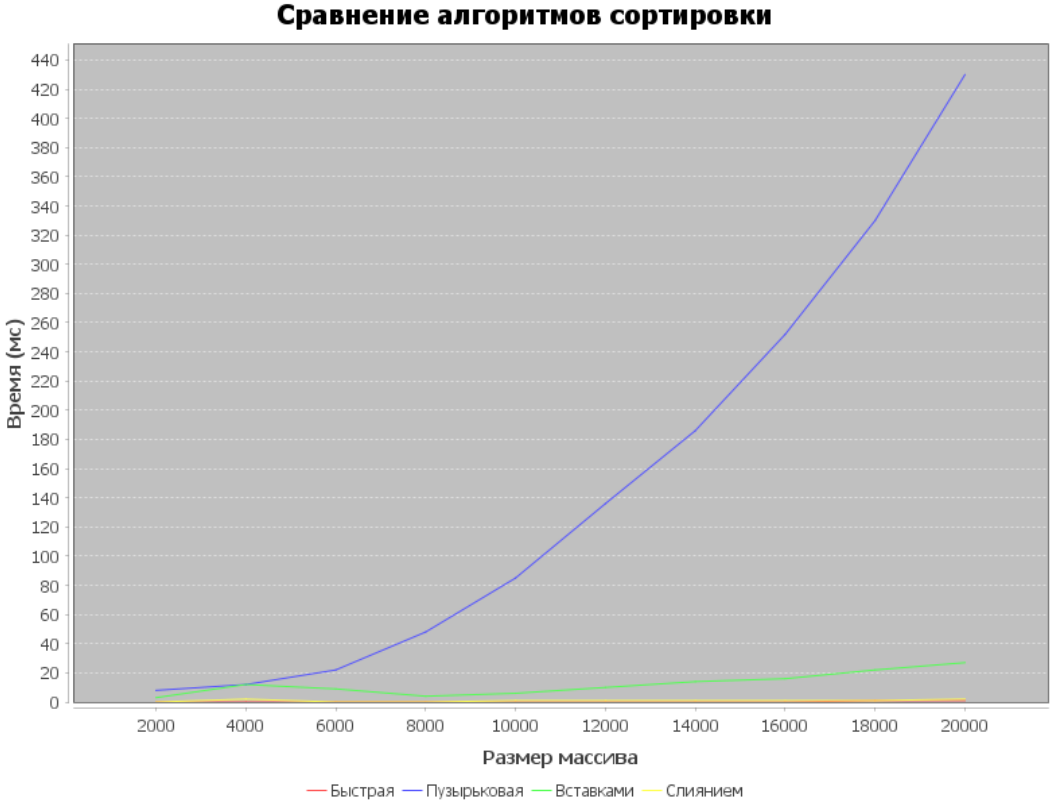
Алгоритм	Размер	Сравнений	Перестановок	Время (мс)	Память (байт)
Пузырьковая	2000	1998000	987654	2300	10560
Вставками	2000	1009800	675423	1200	10512
Быстрая	2000	22600	10534	10	11200
Слиянием	2000	23400	0	18	14200

После сбора данных программа использует библиотеку JFreeChart для построения графиков зависимости времени выполнения от размера массива [1]. На одном графике отображаются линии всех четырёх алгоритмов. Пример графика сохранён в файл sort_performance.png.

Описание построения графика:

- По оси X: количество элементов в массиве
- По оси Y: время выполнения (мс)
- Цветовые линии: разные для каждого алгоритма
- Формат сохранения: PNG

График визуализирует различия в производительности между алгоритмами и наглядно показывает эффективность быстрой и сортировки слиянием по сравнению с пузырьковой и вставками.



Таким образом, была достигнута цель исследования: реализованы и сравнены по ключевым метрикам четыре алгоритма сортировки, а результаты представлены в табличном и графическом виде.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы были реализованы и проанализированы четыре классических алгоритма сортировки: сортировка пузырьком, сортировка вставками, быстрая сортировка и сортировка слиянием. Исследование проводилось на массивах различного размера, для каждого из алгоритмов фиксировались основные метрики: количество сравнений, количество перестановок, время исполнения и объём используемой памяти.

Сортировка пузырьком (Bubble Sort)

Асимптотика по времени составляет $O(n^2)$ во всех случаях, что делает алгоритм крайне неэффективным при работе с большими массивами. По памяти он работает в пределах $O(1)$, так как не требует дополнительных структур. Время исполнения на практике оказалось самым большим среди всех исследованных алгоритмов. Несмотря на низкую производительность, сортировка пузырьком отличается простотой реализации и понимания, что делает её удобной для обучения и демонстрации базовых принципов сортировки [3].

Сортировка вставками (Insertion Sort)

Временная сложность также составляет $O(n^2)$ в среднем и худшем случаях, однако в лучшем случае (почти отсортированные данные) может достигать $O(n)$. Алгоритм не требует дополнительной памяти, работает в пределах $O(1)$. На практике показывает более высокую скорость, чем пузырьковая, особенно при работе с небольшими и частично отсортированными массивами [6]. Реализация немного сложнее, чем у пузырьковой сортировки, но остаётся достаточно простой для понимания и написания.

Быстрая сортировка (Quick Sort)

В среднем работает за $O(n \log n)$, что делает её одной из самых эффективных. Однако в худшем случае (например, при выборе неудачного опорного элемента) может деградировать до $O(n^2)$. Использует $O(\log n)$ памяти за счёт рекурсивных вызовов. На практике показывает отличную производительность даже на больших массивах [5]. Реализация умеренно сложная: требует понимания рекурсии, деления массива и аккуратной работы с индексами.

Сортировка слиянием (Merge Sort)

Гарантированная временная сложность — $O(n \log n)$ во всех случаях, что делает её стабильной по времени. Однако алгоритм требует дополнительной памяти — $O(n)$, так как создаются вспомогательные массивы для слияния. По времени исполнения немного уступает быстрой сортировке, особенно на небольших массивах. Реализация считается одной из самых сложных среди базовых алгоритмов сортировки: требует хорошего понимания рекурсии, копирования данных и управления границами массивов [4]

.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. JFreeChart Documentation [Электронный ресурс]. – Режим доступа: https://www.jfree.org/jfreechart/
2. MSDN. Java Platform API Specification [Электронный ресурс]. – Режим доступа: https://docs.oracle.com/javase/8/docs/api/
3. Wikipedia contributors. Bubble sort. Wikipedia, The Free Encyclopedia. [Электронный ресурс] – Режим доступа: https://en.wikipedia.org/wiki/Bubble_sort
4. Wikipedia contributors. Merge sort. Wikipedia, The Free Encyclopedia. [Электронный ресурс] – Режим доступа: https://en.wikipedia.org/wiki/Merge_sort
5. Wikipedia contributors. Quick sort. Wikipedia, The Free Encyclopedia. [Электронный ресурс] – Режим доступа: https://en.wikipedia.org/wiki/Quicksort
6. Wikipedia contributors. Insertion sort. Wikipedia, The Free Encyclopedia. [Электронный ресурс] – Режим доступа: https://en.wikipedia.org/wiki/Insertion_sort

ПРИЛОЖЕНИЕ

Листинг кода основного файла – Main.java

```
import java.util.*;
import java.io.File;
import org.jfree.chart.ChartFactory;
import org.jfree.chart.JFreeChart;
import org.jfree.chart.ChartUtils;
import org.jfree.data.category.DefaultCategoryDataset;

public class Main {
    public static void main(String[] args) throws Exception {
        System.out.printf("%-15s %-10s %-15s %-15s %-15s %-15s%n",
                "Алгоритм", "Размер", "Сравнений", "Перестановок", "Время (мс)", "Па-
мять (байт)");

        Random random = new Random();

        // Для хранения данных времени: алгоритм -> (размер массива -> время)
        Map<String, Map<Integer, Long>> timeData = new HashMap<>();
        String[] algorithms = {"Пузырьковая", "Вставками", "Быстрая", "Слиянием"};
        for (String alg : algorithms) {
            timeData.put(alg, new LinkedHashMap<>());
        }

        for (int size = 2000; size <= 20000; size += 2000) {
            int[] array = new int[size];
            for (int i = 0; i < size; i++) {
                array[i] = random.nextInt(100_000);
            }

            int[] arrayForBubble = Arrays.copyOf(array, array.length);
            int[] arrayForInsertion = Arrays.copyOf(array, array.length);
            int[] arrayForQuick = Arrays.copyOf(array, array.length);
            int[] arrayForMerge = Arrays.copyOf(array, array.length);

            SortResult bubbleRes = BubbleSort.sort(arrayForBubble);
            bubbleRes.algorithmName = "Пузырьковая";
            SortResult insertionRes = InsertionSort.sort(arrayForInsertion);
            insertionRes.algorithmName = "Вставками";
            SortResult quickRes = QuickSort.sort(arrayForQuick);
            quickRes.algorithmName = "Быстрая";
            SortResult mergeRes = MergeSort.sort(arrayForMerge);
            mergeRes.algorithmName = "Слиянием";

            printResult(bubbleRes);
            printResult(insertionRes);
            printResult(quickRes);
            printResult(mergeRes);
            System.out.println("----------------------------------------------------
--------------------------");

            // Сохраняем время для графика
            timeData.get(bubbleRes.algorithmName).put(size, bubbleRes.timeMs);
            timeData.get(insertionRes.algorithmName).put(size, insertionRes.timeMs);
            timeData.get(quickRes.algorithmName).put(size, quickRes.timeMs);
            timeData.get(mergeRes.algorithmName).put(size, mergeRes.timeMs);
        }
```

```
        createChart(timeData, "Сравнение алгоритмов сортировки", "Размер массива",
"Время (мс)", "sort_performance.png");
        System.out.println("График сохранён в sort_performance.png");
    }

    private static void printResult(SortResult res) {
        System.out.printf("%-15s %-10d %-15d %-15d %-15d %-15d%n",
                res.algorithmName,
                res.arraySize,
                res.comparisons,
                res.swaps,
                res.timeMs,
                res.memoryBytes);
    }

    private static void createChart(Map<String, Map<Integer, Long>> timeData, String
chartTitle,
                                    String xAxisLabel, String yAxisLabel, String
fileName) throws Exception {

        DefaultCategoryDataset dataset = new DefaultCategoryDataset();

        for (var algEntry : timeData.entrySet()) {
            String algorithm = algEntry.getKey();
            for (var sizeEntry : algEntry.getValue().entrySet()) {
                Integer size = sizeEntry.getKey();
                Long time = sizeEntry.getValue();
                dataset.addValue(time, algorithm, size);
            }
        }

        JFreeChart lineChart = ChartFactory.createLineChart(
                chartTitle,
                xAxisLabel,
                yAxisLabel,
                dataset
        );

        int width = 800;
        int height = 600;
        File chartFile = new File(fileName);
        ChartUtils.saveChartAsPNG(chartFile, lineChart, width, height);
    }
}
```

Листинг кода файла вывода результатов измерений – SortResult.java

```
public class SortResult {
    public String algorithmName;
    public int arraySize;
    public long comparisons;
    public long swaps;
    public long timeMs;
    public long memoryBytes;

    public SortResult(String algorithmName, int arraySize, long comparisons, long
swaps, long timeMs, long memoryBytes) {
        this.algorithmName = algorithmName;
        this.arraySize = arraySize;
```

```
        this.comparisons = comparisons;
        this.swaps = swaps;
        this.timeMs = timeMs;
        this.memoryBytes = memoryBytes;
    }
```

Листинг кода реализации пузырьковой сортировки – BubbleSort.java

```
public class BubbleSort {
    public static SortResult sort(int[] array) {
        int n = array.length;
        long comparisons = 0;
        long swaps = 0;

        try {
            System.gc();
            Thread.sleep(50);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        long startMemory = Runtime.getRuntime().totalMemory() -
Runtime.getRuntime().freeMemory();
        long startTime = System.nanoTime();

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                comparisons++;
                if (array[j] > array[j + 1]) {
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                    swaps++;
                }
            }
        }

        long endTime = System.nanoTime();
        long endMemory = Runtime.getRuntime().totalMemory() -
Runtime.getRuntime().freeMemory();

        long elapsedTimeMs = (endTime - startTime) / 1_000_000;
        long memoryUsed = endMemory - startMemory;

        return new SortResult("BubbleSort", n, comparisons, swaps, elapsedTimeMs,
memoryUsed);
    }
}
```

Листинг кода реализации сортировки вставками – InsertionSort.java

```
public class InsertionSort {
    public static SortResult sort(int[] array) {
        int n = array.length;
        long comparisons = 0;
        long swaps = 0;

        try {
            System.gc();
```

```
            Thread.sleep(50);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        long startMemory = Runtime.getRuntime().totalMemory() -
Runtime.getRuntime().freeMemory();
        long startTime = System.nanoTime();

        for (int i = 1; i < n; i++) {
            int key = array[i];
            int j = i - 1;

            while (j >= 0 && array[j] > key) {
                comparisons++;
                array[j + 1] = array[j];
                swaps++;
                j--;
            }
            if (j >= 0) comparisons++; // Последнее сравнение, когда условие не про-
шло

            array[j + 1] = key;
        }

        long endTime = System.nanoTime();
        long endMemory = Runtime.getRuntime().totalMemory() -
Runtime.getRuntime().freeMemory();

        long elapsedTimeMs = (endTime - startTime) / 1_000_000;
        long memoryUsed = endMemory - startMemory;

        return new SortResult("InsertionSort", n, comparisons, swaps, elapsedTimeMs,
memoryUsed);
    }
}
```

Листинг кода реализации быстрой сортировки – QuickSort.java

```
public class QuickSort {
    private static long comparisons = 0;
    private static long swaps = 0;

    public static SortResult sort(int[] array) {
        comparisons = 0;
        swaps = 0;

        try {
            System.gc();
            Thread.sleep(50);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        long startMemory = Runtime.getRuntime().totalMemory() -
Runtime.getRuntime().freeMemory();
        long startTime = System.nanoTime();

        quickSort(array, 0, array.length - 1);
```

```
        long endTime = System.nanoTime();
        long endMemory = Runtime.getRuntime().totalMemory() -
Runtime.getRuntime().freeMemory();

        long elapsedTimeMs = (endTime - startTime) / 1_000_000;
        long memoryUsed = endMemory - startMemory;

        return new SortResult("QuickSort", array.length, comparisons, swaps,
elapsedTimeMs, memoryUsed);
    }

    private static void quickSort(int[] array, int low, int high) {
        if (low < high) {
            int pi = partition(array, low, high);
            quickSort(array, low, pi - 1);
            quickSort(array, pi + 1, high);
        }
    }

    private static int partition(int[] array, int low, int high) {
        int pivot = array[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            comparisons++;
            if (array[j] <= pivot) {
                i++;
                swap(array, i, j);
            }
        }
        swap(array, i + 1, high);
        return i + 1;
    }

    private static void swap(int[] array, int i, int j) {
        swaps++;
        int temp = array[i];
        array[i] = array[j];
        array[j] = temp;
    }
}
```

Листинг кода реализации сортировки слиянием – MergeSort.java

```
public class MergeSort {
    private static long comparisons = 0;

    public static SortResult sort(int[] array) {
        comparisons = 0;

        try {
            System.gc();
            Thread.sleep(50);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        long startMemory = Runtime.getRuntime().totalMemory() -
Runtime.getRuntime().freeMemory();
        long startTime = System.nanoTime();
```

```
        mergeSort(array, 0, array.length - 1);

        long endTime = System.nanoTime();
        long endMemory = Runtime.getRuntime().totalMemory() -
Runtime.getRuntime().freeMemory();

        long elapsedTimeMs = (endTime - startTime) / 1_000_000;
        long memoryUsed = endMemory - startMemory;

        // В слиянии перестановок как таковых нет, только копирование, считаем
swaps=0
        return new SortResult("MergeSort", array.length, comparisons, 0,
elapsedTimeMs, memoryUsed);
    }

    private static void mergeSort(int[] array, int left, int right) {
        if (left < right) {
            int mid = (left + right) / 2;
            mergeSort(array, left, mid);
            mergeSort(array, mid + 1, right);
            merge(array, left, mid, right);
        }
    }

    private static void merge(int[] array, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] leftArr = new int[n1];
        int[] rightArr = new int[n2];

        System.arraycopy(array, left, leftArr, 0, n1);
        System.arraycopy(array, mid + 1, rightArr, 0, n2);

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            comparisons++;
            if (leftArr[i] <= rightArr[j]) {
                array[k++] = leftArr[i++];
            } else {
                array[k++] = rightArr[j++];
            }
        }

        while (i < n1) {
            array[k++] = leftArr[i++];
        }

        while (j < n2) {
            array[k++] = rightArr[j++];
        }
    }
}
```